

Compte Rendu de Projet Développement d'une application web de clavardage en ligne

Bubbly



Sommaire :

PRESENTATION	3
THEME	3
OBJECTIFS	3
ANALYSE DU SUJET	3
FONCTIONNALITES ATTENDUES	3
METHODOLOGIE	4
APPROCHE	4
REPARTITION DU TRAVAIL	4
OUTILS ET RESSOURCES	4
REALISATION	4
ARCHITECTURE DE L'APPLICATION	4
DESCRIPTION DES FICHIERS	5
ANALYSE DETAILLEE DU CODE	6
SYNTHESE	8
BILAN GENERAL	8
DIFFICULTES RENCONTREES	8
LIMITES IDENTIFIEES	8
AMELIORATIONS POSSIBLES	8
CONCLUSION	9

Présentation

Thème

Ce projet consiste à réaliser une application de clavardage en ligne, que j'ai appelé Bubbly, en utilisant les langages du web : HTML, CSS, PHP et JavaScript. Développé dans le cadre du cours de Projet Transversal Web, il vise à mettre en pratique les notions suivantes :

- Développement d'un back-end PHP avec gestion de sessions
- Connexion à une base de données relationnelle MySQL via PDO
- Gestion des utilisateurs.
- Création et administration de salons de discussion
- Persistance des messages en base de données
- Protection des routes par vérification de session

Objectifs

L'objectif de ce projet est de livrer une application web de messagerie fonctionnelle permettant à plusieurs utilisateurs de s'inscrire, de se connecter et d'échanger des messages au sein de salons dédiés. Le projet s'appuie sur une architecture procédurale PHP avec une séparation claire entre les pages de rendu front et les scripts de traitement back. Il nous a été spécifié que pour ce projet la partie Back est la plus importante, le Front pouvait rester simple et généré par IA si nécessaire.

L'objectif pédagogique est de concevoir une application web complète en partant de zéro : de la modélisation de la base de données jusqu'à l'interface utilisateur, en passant par la sécurisation des accès et la gestion des erreurs.

Analyse du sujet

Fonctionnalités attendues

Le sujet impose la réalisation d'une application de clavardage permettant :

1. Création d'un compte avec identifiant et mot de passe (utilisateur et administrateur).
2. Création de salons (zones d'échange de messages).
3. Visibilité des salons en fonction des utilisateurs (membres invités).
4. Échange de messages en « temps réel ».

La persistance des données doit être assurée soit par une base de données relationnelle, soit par des fichiers JSON. Notre groupe a opté pour une base de données MySQL. Pour aller plus loin, le sujet recommande l'utilisation de la

programmation orientée objet, de la réécriture d'URL, et l'ajout de JavaScript pour la mise à jour dynamique de la conversation.

Méthodologie

Approche

- Développement itératif par étapes, conformément au plan détaillé du sujet :
- Mise en place de l'environnement (VM, PhpStorm, GitHub).
- Faire un front simple avec les fonctionnalités de base
- Commencer à intégrer du back
 - Développement des pages de connexion et d'inscription.
 - Développement des salons : création, modification, affichage.
 - Développement de l'envoi et de l'affichage des messages.
- Tests fonctionnels de l'ensemble de l'application.

Répartition du travail

Le projet a été réalisé en groupe. Nous nous sommes répartis les pages et fonctionnalités à développer, puis nous avons organisé des sessions de fusion et de tests communs pour garantir la cohérence de l'ensemble. Le dépôt GitHub partagé (marylou014/bubbly) a servi d'outil de versionnage et de collaboration tout au long du projet.

Outils et Ressources

- Langages : PHP, HTML, CSS, JavaScript
 - Base de données : MySQL
 - Environnement : VM en local, PhpStorm
 - Versionnage : GitHub (<https://github.com/marylou014/bubbly>)
 - Documentation : documentation officielle PHP, et vidéo YouTube
-

Réalisation

Architecture de l'application

L'application suit une architecture web classique, sans framework. Chaque page PHP cumule le rôle de vue (rendu HTML) et de contrôleur (traitement des données). La persistance est déléguée à MySQL via PDO. Le projet est organisé en fichiers à plat dans un unique répertoire racine.

Description des fichiers

Fichier	Rôle
index.php	Page d'accueil : présentation de l'application, boutons S'inscrire / Se connecter.
create_compte.php	Formulaire d'inscription : saisie du login et du mot de passe.
traitement.php	Script back : insertion d'un nouvel utilisateur en base de données.
connexion.php	Formulaire de connexion : vérification identifiant/mot de passe en BDD, création de session.
vérification.php	Middleware de session : vérifie si l'utilisateur est connecté, redirige vers index.php sinon.
salon.php	Liste des salons accessibles à l'utilisateur connecté.
create_salon.php	Formulaire de création d'un salon (nom, description, membre invité).
traitement_salon.php	Script back : insertion du salon en BDD, redirection vers salon.php.
modif_salon.php	Formulaire de modification d'un salon existant.
conversation.php	Affichage des messages d'un salon et formulaire d'envoi d'un nouveau message.
deconnexion.php	Destruction de la session et redirection vers index.php.
connection_bdd.php	Connexion PDO à la base de données MySQL (inclus dans les autres scripts).

Analyse détaillée du code

1. Connexion à la base de données (connection_bdd.php)

La connexion à la base de données est centralisée dans un fichier dédié, inclus dans chaque script qui en a besoin. Elle utilise PDO (PHP Data Objects), ce qui offre une interface unifiée et sécurisée pour interroger MySQL. Le mode d'erreur `ERRMODE_EXCEPTION` est activé pour lever des exceptions en cas de problème.

```
$base = new PDO("mysql:host=$servername;dbname=clavarde",  
$username_base, $password);
```

2. Protection des routes (vérification.php)

Un fichier `vérification.php` est inclus en tête de chaque page protégée via `require`. Il vérifie la présence du cookie de session `PHPSESSID`, démarre la session, puis contrôle que la variable `$_SESSION["connecté"]` est bien définie et vraie. Si ce n'est pas le cas, l'utilisateur est redirigé vers la page d'accueil. Ce mécanisme joue le rôle de garde-fou centralisé.

```
if(!isset($_SESSION["connecté"]) || !$_SESSION["connecté"]) {  
header("Location: index.php"); exit; }
```

3. Authentification (connexion.php)

La page de connexion traite le formulaire POST en deux étapes : vérification que les champs ne sont pas vides, puis requête SQL préparée pour comparer l'identifiant et le mot de passe avec la table `users`. L'utilisation de `prepare()` et `execute()` avec des paramètres liés protège contre les injections SQL. En cas de succès, une session est créée et l'utilisateur est redirigé vers la liste des salons.

```
$requete = $base->prepare("SELECT * FROM users WHERE username =  
? AND password = ?");
```

4. Création de salon (create_salon.php + traitement_salon.php)

Le formulaire de création d'un salon collecte trois informations : le nom, la description et un membre invité. Le script de traitement vérifie que les champs obligatoires ne sont pas vides, puis insère une ligne dans la table `salon` avec les valeurs liées. Le champ `auteur` est automatiquement rempli depuis la session (`$_SESSION["username"]`).

```
$requete = $base->prepare("INSERT INTO salon VALUES (0, :nom,  
:description, :auteur, :membre)");
```

5. Affichage et envoi de messages (conversation.php)

La page de conversation récupère l'identifiant du salon depuis l'URL (`$_GET['id_salon']`) et charge l'ensemble des messages correspondants, triés par date et identifiant. Chaque message est affiché avec le nom de l'auteur (en couleur) et l'heure d'envoi. L'envoi d'un nouveau message est géré par un formulaire POST dans la même page : lorsque le bouton est cliqué, le message est inséré en base avec l'heure courante via `CURRENT_TIME()`.

```
$requete = $base->prepare("SELECT * FROM message WHERE  
id_salon_mess = ? ORDER BY date, id_mess");
```

```
$req->execute(["id_salon_mess" => $id_salon, "message" => $msg,  
"auteur" => $username]);
```

Tests

Les tests ont été effectués manuellement en naviguant dans l'application avec différents comptes utilisateur. Les scénarios couverts sont les suivants :

- Inscription avec des identifiants valides et invalides (champs vides).
- Connexion avec un compte existant, puis avec des identifiants incorrects.
- Accès direct à une page protégée sans session active (vérification de la redirection).
- Création d'un salon avec et sans remplissage des champs obligatoires.
- Modification d'un salon existant.
- Envoi de messages dans un salon et vérification de leur affichage ordonné.
- Déconnexion et vérification de la destruction de la session.

Synthèse

Bilan général

Le projet Bubbly remplit ses objectifs principaux : l'application est fonctionnelle et couvre l'ensemble des étapes du plan détaillé du sujet. Les fonctionnalités de création de compte, de connexion sécurisée, de gestion des salons et d'échange de messages sont opérationnelles. La base de données MySQL est correctement utilisée pour la persistance de toutes les données.

Les bonnes pratiques de développement ont été respectées : utilisation de requêtes préparées PDO pour la sécurité, centralisation de la connexion BDD et de la vérification de session dans des fichiers dédiés, séparation des formulaires et des scripts de traitement, et commentaires dans le code.

Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées lors du développement :

- La répartition du travail, certaines personnes ont fait beaucoup plus que d'autres...
- La gestion des sessions PHP entre plusieurs pages : comprendre le rôle du cookie PHPSESSID et la nécessité d'appeler `session_start()` avant toute écriture de headers.
- La configuration de l'environnement local (VM) et les problèmes de droits d'accès à la base de données.
- Le passage de données entre pages via `$_GET` (id_salon dans l'URL) et la sécurisation de ces paramètres.
- L'affichage dynamique des messages sans rechargement de page (point bonus non complètement abouti).

Limites identifiées

Le projet dans sa forme actuelle présente quelques limites à prendre en considération :

- Les mots de passe sont stockés en clair dans la base de données. Une solution robuste utiliserait `password_hash()` et `password_verify()` de PHP.
- La gestion des membres d'un salon est limitée à un seul champ texte libre.
- L'absence de validation et d'assainissement des entrées utilisateur côté serveur expose potentiellement l'application à des attaques.
- Il n'y a pas de mise à jour automatique de la conversation (le bonus JavaScript n'a pas été implémenté) : l'utilisateur doit recharger la page pour voir les nouveaux messages.
- Les grilles de salons sont accessibles à tous les utilisateurs connectés, sans filtrage strict par appartenance.

Améliorations possibles

Plusieurs axes d'amélioration pourraient enrichir ce projet dans une version future :

- Hachage des mots de passe avec `password_hash()` / `password_verify()` pour renforcer la sécurité.

- Mise à jour en temps réel de la conversation via JavaScript, conformément au bonus du sujet.
- Gestion des membres d'un salon via une table de liaison pour permettre plusieurs membres par salon.
- Interface d'administration dédiée pour la gestion des comptes et des salons.
- CLI (ligne de commande) pour la création d'un compte administrateur, comme prévu dans le plan détaillé du sujet.
- Réécriture d'URL avec un fichier .htaccess pour des adresses plus propres et plus sécurisées.

Conclusion

Ce projet transversal constitue une application concrète et complète des langages du web vus en cours de Bachelor 1. Il démontre qu'il est possible de développer une application de messagerie interactive en PHP pur, sans framework, en s'appuyant sur une base de données relationnelle MySQL et une gestion de sessions native.

La démarche adoptée — analyse du sujet, modélisation de la base de données, développement par étapes, tests fonctionnels — s'est révélée efficace et structurante. Les difficultés rencontrées, notamment sur la gestion des sessions et la coordination en groupe, ont été un bon apprentissage.

Bubbly pose ainsi les bases d'une messagerie web opérationnelle, avec des perspectives d'évolution claires vers plus de sécurité, de dynamisme et de richesse fonctionnelle.